

# DBMS Exam Notes: Serializability

Transaction Scheduling Basics • Parallel vs. Serial Equivalence • GATE Review

## 1. WHAT IS SERIALIZABILITY?

**Serializability** is the benchmark criterion used to check whether a *parallel (concurrent) schedule* can yield the exact same database state as some non-interleaved *serial schedule*.

Parallel / Concurrent Schedule



Equivalent to Any Serial Schedule?



YES: SERIALIZABLE

### Super-Easy Memory Formula

**S → E → S**: Schedule Equivalent to Serial. If an interleaved execution gives the same outcome as a step-by-step serial execution, it is safe and serializable!

## 2. SCHEDULE TYPES: SERIAL VS. PARALLEL

### Serial Schedule (No Interleaving)

One transaction completely finishes before another starts.

```
-- Order: T1 -> T2 T1: READ(A),  
WRITE(A), READ(B), WRITE(B) T2: READ(A),  
WRITE(A) -- Order: T2 -> T1 T2: READ(A),  
WRITE(A) T1: READ(A), WRITE(A), READ(B),  
WRITE(B)
```

### Parallel / Concurrent Schedule

Operations of multiple transactions are interleaved.

```
-- Interleaved Execution T1:  
READ(A) T2: READ(A) T1: WRITE(A) T2:  
WRITE(A) (T1 has not completed before T2  
begins)
```

## 3. PRECEDENCE GRAPH REPRESENTATION

In serial schedules, execution flow moves directly from one transaction node to another without overlaps:

Order T1 → T2

[T1] → [T2]

Order T2 → T1

[T2] → [T1]

## 4. TYPES OF SERIALIZABILITY TESTING

To determine if an interleaved parallel schedule is serializable, two main testing techniques are used:

### 1 Conflict Serializability

Analyzes **conflicting operations** (Read-Write, Write-Read, Write-Write) on the same data item across transactions and uses a **Precedence Graph (Cycle Test)**.

### 2 View Serializability

A broader condition that tests whether a schedule is **view equivalent** to a serial schedule by checking initial reads, updated writes, and final reads.

## 5. FORMULA: POSSIBLE SERIAL ORDERS (HIGH-YIELD EXAM POINT)

For a set of  $n$  concurrent transactions, the total number of distinct possible serial execution orders is given by:

**Possible Serial Orders =  $n!$**

(where  $n$  = number of transactions in the schedule)

| Number of Transactions ( $n$ ) | Formula ( $n!$ )                    | Total Possible Serial Orders | Explicit Serial Orders                                                              |
|--------------------------------|-------------------------------------|------------------------------|-------------------------------------------------------------------------------------|
| 2 Transactions (T1, T2)        | $2! = 2 \times 1$                   | 2                            | 1. T1 → T2<br>2. T2 → T1                                                            |
| 3 Transactions (T1, T2, T3)    | $3! = 3 \times 2 \times 1$          | 6                            | 1. T1→T2→T3   2. T1→T3→T2<br>3. T2→T1→T3   4. T2→T3→T1<br>5. T3→T1→T2   6. T3→T2→T1 |
| 4 Transactions (T1..T4)        | $4! = 4 \times 3 \times 2 \times 1$ | 24                           | 24 distinct permutations                                                            |
| 5 Transactions (T1..T5)        | $5!$                                | 120                          | 120 distinct permutations                                                           |

## 6. SERIAL VS. PARALLEL VS. SERIALIZABLE COMPARISON

| Feature      | Serial Schedule               | Parallel Schedule      | Serializable Schedule                  |
|--------------|-------------------------------|------------------------|----------------------------------------|
| Execution    | One transaction after another | Interleaved operations | Interleaved operations                 |
| Interleaving | Forbidden ❌                   | Allowed ✅              | Allowed ✅                              |
| Needs Test?  | No (Always Serial)            | Yes                    | Result of successful testing           |
| Equivalence  | Self-evident                  | May or may not be safe | Equivalent to some $n!$ serial order ✅ |

## 7. CORE CONCEPT FLOW & EXAM TRAPS

### Decision Tree for Given Schedule:

- Is there interleaving between operations?
  - **NO** → Already a **Serial Schedule**.
  - **YES** → It is a **Parallel Schedule**.
- For Parallel Schedule: Can we find an equivalent serial order using Conflict/View testing?
  - **YES** → **Serializable Schedule** (Safe to execute).
  - **NO** → **Non-Serializable Schedule** (Risk of database inconsistency).

### ⚠️ Exam Trap: Serial Schedule ≠ Serializable Schedule

- A **Serial Schedule** has zero interleaving.
- A **Serializable Schedule** IS interleaved, but its outcome matches a non-interleaved serial schedule.
- **Golden Rule:** Interleaving is allowed in serializable schedules as long as correctness is preserved!

### ⚡ 30-Second Revision Snapshot

- **Serializable Formula:** Parallel Schedule → Equivalent Serial Schedule
- **Order Calculation:**  $n!$  serial orders for  $n$  transactions
- **Two Main Testing Methods:** Conflict Serializability & View Serializability